



دانشگاه صنعتی امیر کبیر
(پلی تکنیک تهران)

Computer Architecture

Data Representation

Afarghadan@aut.ac.ir

عظیم فرقدان 

- 1 Data types**
- 2 Complements**
- 3 Fixed point Representations**
- 4 Floating point representations**

- Data are numbers and other binary-coded information that are operated on
- Possible data types in registers:
 - Numbers used in computations
 - Letters of the alphabet used in data processing
 - Other discrete symbols used for specific purposes
- All types of data, except binary numbers, are represented in binary-coded form.
- This is because register are made up of Flip-Flops and Flip-Flops are two-state devices that can store only 1's and 0's.

- A number system of base, or radix, r is a system that uses distinct symbols for r digits. Numbers are represented by a string of digit symbols .

- Conversion of Binary to decimal: 101101

$$1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 45$$

$$(101101)_2 = (45)_{10}$$

- Conversion of Octal to decimal : (736.4)₈

$$7 \times 8^2 + 3 \times 8^1 + 6 \times 8^0 + 4 \times 8^{-1} = (478.5)_{10}$$

- Conversion of hexadecimal to octal: (F3)₁₆

$$F \times 16^1 + 3 \times 16^0 = (243)_{10}$$

Conversion of Decimal to binary

- Conversion from decimal to radix r system is carried out by separating the number into its integer and fraction parts and converting each part separately

Figure 3-1 Conversion of decimal 41.6875 into binary.

Integer = 41

41	
20	1
10	0
5	0
2	1
1	0
0	1

$$(41)_{10} = (101001)_2$$

Fraction = 0.6875

0.6875
2
1.3750
x 2
0.7500
x 2
1.5000
x 2
1.0000

$$(0.6875)_{10} = (0.1011)_2$$

$$(41.6875)_{10} = (101001.1011)_2$$

- Each octal digit corresponds to three binary digits
- Each hexadecimal digit corresponds to four binary digits
- Rather than specifying numbers in binary form, refer to them in octal or hexadecimal and reduce the number of digits by $1/3$ or $1/4$, respectively.

<u>1</u>	<u>2</u>	<u>7</u>	<u>5</u>	<u>4</u>	<u>3</u>	Octal
1 0 1 0	1 1 1 1	0 1 1 0	0 0 1 1			Binary
A	F	6	3			Hexadecimal

Figure 3-2 Binary, octal, and hexadecimal conversion.

Octal to Hexadecimal

TABLE 3-2 Binary-Coded Hexadecimal Numbers

Hexadecimal number	Binary-coded hexadecimal	Decimal equivalent
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
A	1010	10
B	1011	11
C	1100	12
D	1101	13
E	1110	14
F	1111	15
14	0001 0100	20
32	0011 0010	50
63	0110 0011	99
F8	1111 1000	248

Code
for one
hexadecimal
digit

TABLE 3-1 Binary-Coded Octal Numbers

Octal number	Binary-coded octal	Decimal equivalent
0	000	0
1	001	1
2	010	2
3	011	3
4	100	4
5	101	5
6	110	6
7	111	7
10	001 000	8
11	001 001	9
12	001 010	10
24	010 100	20
62	110 010	50
143	001 100 011	99
370	011 111 000	248

Code
for one
octal
digit

- Numerous different codes can be obtained by arranging four bits in 10 distinct combinations.
- Binary-coded decimal (BCD) is a class of binary encodings of decimal numbers where each digit is represented by a fixed number of bits, usually four or eight.
- Sometimes, special bit patterns are used for a sign or other indications (E.g: error or overflow). This particular code is called binary coded decimal and commonly know as BCD.
- A binary code is a group of n bits that assume up to 2^n distinct combinations

Decimal Representation

TABLE 3-3 Binary-Coded Decimal (BCD) Numbers

Decimal number	Binary-coded decimal (BCD) number	
0	0000	Code for one decimal digit
1	0001	
2	0010	
3	0011	
4	0100	
5	0101	
6	0110	
7	0111	
8	1000	
9	1001	
10	0001 0000	
20	0010 0000	
50	0101 0000	
99	1001 1001	
248	0010 0100 1000	

TABLE 3-2 Binary-Coded Hexadecimal Numbers

Hexadecimal number	Binary-coded hexadecimal	Decimal equivalent	
0	0000	0	Code for one hexadecimal digit
1	0001	1	
2	0010	2	
3	0011	3	
4	0100	4	
5	0101	5	
6	0110	6	
7	0111	7	
8	1000	8	
9	1001	9	
A	1010	10	
B	1011	11	
C	1100	12	
D	1101	13	
E	1110	14	
F	1111	15	
14	0001 0100	20	
32	0011 0010	50	
63	0110 0011	99	
F8	1111 1000	248	

- The standard alphanumeric binary code is ASCII
- This uses seven bits to code 128 characters
- Binary codes are required since registers can hold binary information only

TABLE 3-4 American Standard Code for Information Interchange (ASCII)

Character	Binary code	Character	Binary code
A	100 0001	0	011 0000
B	100 0010	1	011 0001
C	100 0011	2	011 0010
D	100 0100	3	011 0011
E	100 0101	4	011 0100
F	100 0110	5	011 0101
G	100 0111	6	011 0110
H	100 1000	7	011 0111
I	100 1001	8	011 1000
J	100 1010	9	011 1001

- Complements are used in digital computers for simplifying the subtraction operation and for logical manipulation.
- There are two types of complements for each base r system:
 - R 's complement (2's and 10's complements)
 - $(r-1)$'s complement (1's and 9's complements)
- When the value of the base r is substituted in the name, the two types are referred to as the 2's and 1's complement for binary numbers and the 10's and 9's complement for decimal numbers.

- **9's Complement:** Given a number N in base r having n digits, the $(r - 1)$'s complement of N is defined as $(r^n - 1) - N$.
 - For decimal, the 9's complement of N is $(10^n - 1) - N$.

For Example,

- The 9's complement of 546700 is $999999 - 546700 = 453299$
- The 9's complement of 453299 is $999999 - 453299 = 546700$

- **1's complement:** For binary, the 1's complement of N is $(2^n - 1) - N$.

For example,

- The 1's complement of 1011001 is $1111111 - 1011001 = 0100110$

- **10's and 2's complement** : The r 's complement of an n -digit number N in base r is defined as $r^n - N$.
- This is the same as adding 1 to the $(r - 1)$'s complement
- **For example:**
 - The 10's complement of 2389 is $7610 + 1 = 7611$
 - The 2's complement of 101100 is $010011 + 1 = 010100$

- The subtraction of two n digit unsigned numbers $M - N$ ($N \neq 0$) in base r can be done as follows:
 1. Add the minuend M to the r 's complement of the subtrahend N . This performs $M + (r^n - N) = M - N + r^n$.
 2. If $M \geq N$, the sum will produce an end carries r^n , which is discarded, and what is left is the result $M - N$.
 3. If $M < N$, the sum does not produce an end carry and is equal to $r^n - (N - M)$, which is the r 's complement of $(N - M)$. To obtain the answer in a familiar form, take the r 's complement of the sum and place a negative sign in front

Sample Subtraction

Subtraction with complements is done with binary numbers in a similar manner using the same procedure outlined above. Using the two binary numbers $X = 1010100$ and $Y = 1000011$, we perform the subtraction $X - Y$ and $Y - X$ using 2's complements:

$$\begin{array}{r} X = 1010100 \\ 2's \text{ complement of } Y = +0111101 \\ \hline \text{Sum} = 10010001 \\ \text{Discard end carry } 2^7 = -10000000 \\ \hline \text{Answer: } X - Y = 0010001 \end{array}$$
$$\begin{array}{r} Y = 1000011 \\ 2's \text{ complement of } X = +0101100 \\ \hline \text{Sum} = 1101111 \end{array}$$

There is no end carry

Answer is negative $0010001 = 2's \text{ complement of } 1101111$

- Positive integers and zero can be represented by unsigned numbers
- Signed numbers have msb(most significant bit) as 0 for positive and 1 for negative – msb is the sign bit
- **Fixed point position** usually uses one of the two following positions
 - A binary point in the extreme left of the register to make it a fraction
 - A binary point in the extreme right of the register to make it an integer
 - In both cases, a binary point is not actually present
- The **floating-point representations** uses a second register to designate the position of the binary point in the first register

- When an integer is positive, the msb, or sign bit, is 0 and the remaining bits represent the magnitude
- When an integer is negative, the msb, or sign bit, is 1, but the rest of the number can be represented in one of three ways
 - **Signed-magnitude representation**
 - **Signed-1's complement representation**
 - **Signed-2's complement representation**
- +14 : The only way to represent it is 0000 1110 (8 bits)
- -14 : Signed magnitude: 1 0001110
Signed 1's complement: 1 1110001
Signed 2's complement: 1 1110010

- Addition of two signed 2's complement numbers does not require a comparison or subtraction – only addition and complementation
 - Add the two numbers, including their sign bits
 - Discard any carry out of the sign bit position
 - All negative numbers must be in the 2's complement form
 - If the sum obtained is negative, then it is in 2's complement form.
- Example:

$$\begin{array}{r} -6 \quad 1111010 \\ +13 \quad 0001101 \\ \hline +7 \quad 0000111 \end{array}$$

- Subtraction of two signed 2's complement numbers is as follows
 - Take the 2's complement form of the subtrahend (including sign bit)
 - Add it to the minuend (including the sign bit)
 - A carry out of the sign bit position is discarded
- Example:

$$\begin{array}{r} -6 \quad 1111010 \\ -13 \quad 11110011 \\ \hline -19 \quad 11101101 \end{array}$$

- An overflow occurs when two numbers of n digits each are added and the sum occupies $n + 1$ digits
- Overflows are problems since the width of a register is finite
- Therefore, a flag is set if this occurs and can be checked by the user
- Detection of an overflow depends on if the numbers are signed or unsigned
- For unsigned numbers, an overflow is detected from the end carry out of the msb
- For addition of signed numbers, an overflow cannot occur if one is positive and one is negative – both have to have the same sign
- An overflow can be detected if the carry into the sign bit position and the carry out of the sign bit position are not equal
- Example:

carries:	0	1
+70	0	1000110
+80	0	1010000
<u>+150</u>	<u>1</u>	<u>0010110</u>

carries:	1	0
-70	1	0111010
-80	1	0110000
<u>-150</u>	<u>0</u>	<u>1101010</u>

- Representation of signed decimal numbers in BCD is similar to the representation of signed numbers in binary.
- The sign of a number is represented with four bits
0000 for +
1001 for –
- To obtain the 10's complement of a BCD number, first take the 9's complement and then add one to the least significant digits
- Example: $(+375) + (-240) = +135$ done in signed 10's Complement System

$$\begin{array}{rcl} 0\ 375 & (0000\ 0011\ 0111\ 0101)_{\text{BCD}} \\ +9\ 760 & (1001\ 0111\ 0110\ 0000)_{\text{BCD}} \\ \hline 0\ 135 & (0000\ 0001\ 0011\ 0101)_{\text{BCD}} \end{array}$$

Floating Point Representation

- The floating-point representation of a number has two parts
- The first part represents a signed, fixed-point number – the mantissa
- The second part designates the position of the binary point – the exponent.
- For Example, the decimal number +6132.789 is represented in floating point with a fraction and an exponent as follows:

<i>Fraction</i>	<i>Exponent</i>
+0.6132789	+04

Equivalent to $+0.6132789 \times 10^{+4}$

- A floating-point number is said to be normalized if the most significant digit of the mantissa is nonzero
- The decimal number 350 is normalized, 00350 is not
- The 8-bit number 00011010 is not normalized
 - Normalize it by fraction = 11010000 and exponent = -3
- Normalized numbers provide the maximum possible precision for the floating-point number
- Example: the binary number +1001.11 (with 8-bit fraction and 6-bit exponent)
 - Fraction: 01001110
 - Exponent: 000100
 - Equivalent to $+(.1001110)_2 \times 2^{+4}$



Thanks
